

Operating Systems Project

Phase 2 Report

Group Members:

Yixuan Ma ym3116

Uditi Sharma us2133

1 Architecture and Design

High-Level Structure

Phase 2 extends the Phase 1 shell into a **client-server remote shell** using TCP socket communication. The system is split into two independent programs:

- **myshell_server** — runs on the remote Linux machine, receives commands over a socket, executes them using the Phase 1 shell engine, and sends the output back.
- **myshell_client** — runs on the user's machine, presents the familiar **\$** prompt, sends typed commands to the server, and displays the received output.

The flow of a single command is:

User input → client sends command → server receives → executes → captures output → sends back → client displays

The Phase 1 standalone shell (**myshell**) is preserved unchanged and still fully functional.

2 Implementation Highlights

1. TCP Socket Setup (Server)

The server creates a TCP socket, sets `SO_REUSEADDR` to avoid port-in-use errors on restart, binds to all interfaces, and listens for connections:

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
listen(server_fd, BACKLOG);
```

The server then loops on `accept()`, serving one client at a time before waiting for the next.

2. Hostname Resolution (Client)

The client uses `getaddrinfo()` instead of `inet_pton()` so it accepts both IP addresses and hostnames:

```
getaddrinfo(host, port_str, &hints, &res);
connect(sock, rp->ai_addr, rp->ai_addrlen);
```

This makes the client flexible — it works with 127.0.0.1 locally or a remote hostname like myserver.nyuad.nyu.edu.

3. Output Capture via Pipe + Fork

The core of the server is capturing a command's output without printing it to the terminal. A pipe is created, and a child process redirects both stdout and stderr into the write end before executing the Phase 1 pipeline. The parent reads from the read end:

```
pipe(pipe_fds);
if (fork() == 0) {
    dup2(pipe_fds[1], STDOUT_FILENO); // stdout → pipe
    dup2(pipe_fds[1], STDERR_FILENO); // stderr → pipe
    close(pipe_fds[0]); close(pipe_fds[1]);
    execute_pipeline(pl);
    exit(0);
}
close(pipe_fds[1]); // parent closes write end to see EOF
// parent reads from pipe_fds[0] into buffer
```

The parent closes the write end before reading — this is critical, otherwise `read()` blocks forever waiting for EOF that never comes.

4. Length-Prefix Framing Protocol

TCP has no message boundaries, so raw `send()/recv()` calls cannot tell where one command's output ends and the next begins. Every response is prefixed with a 4-byte length in network byte order:

```
// Server sends:
uint32_t net_len = htonl((uint32_t)out_len);
send_all(client_fd, &net_len, sizeof(net_len));
send_all(client_fd, output, out_len);

// Client receives:
recv_all(sock, &net_len, sizeof(net_len));
```

```
uint32_t out_len = ntohl(net_len);
```

[htonl/ntohl](#) convert between host and network byte order, ensuring correctness across different machine architectures.

3 Execution Instructions

Compile

make clean && make

This produces three binaries: [myshell](#) (Phase 1), [myshell_server](#), and [myshell_client](#). (Phase 2)

Run

Terminal 1 — Start the server (on the remote Linux machine):

```
./myshell_server <port>
```

Example:

```
./myshell_server 9090
```

Terminal 2 — Start the client (from any machine):

```
./myshell_client <server_host> <port>
```

Example (same machine):

```
./myshell_client 127.0.0.1 9090
```

Example (remote server):

```
./myshell_client 192.168.1.10 9090
```

4 Testing

Test Case 1: Basic Command Execution

```
$ ls
```

```
Makefile include myshell myshell_client myshell_server src
```

Server output:

```
[INFO] Server started, waiting for client connections...
```

```
[INFO] Client connected.
```

```
[RECEIVED] Received command: "ls" from client.
```

```
[EXECUTING] Executing command: "ls"
```

```
[OUTPUT] Sending output to client:
```

```
Makefile include myshell myshell_client myshell_server src
```

Test Case 2: Pipe Command

```
$ ls | grep src  
src
```

Server shows [RECEIVED], [EXECUTING], and [OUTPUT] messages correctly.

Test Case 3: Command Not Found

```
$ unknowncmd  
Command not found.
```

Server output:

```
[RECEIVED] Received command: "unknowncmd" from client.  
[EXECUTING] Executing command: "unknowncmd"  
[ERROR] Command not found: "unknowncmd"  
[OUTPUT] Sending error message to client: "Command not found."
```

Test Case 4: Parse Error (Empty Pipe)

```
$ ls |  
Command missing after pipe.
```

Server shows [ERROR] Parse error: Command missing after pipe.

Test Case 5: Exit Command

```
$ exit  
[Client] Disconnected from server.
```

Server shows [INFO] Client requested exit.

5 Challenges

1. **Server Output Formatting** — Matching the exact output format from the specification ([INFO], [RECEIVED], [EXECUTING], [OUTPUT], [ERROR]) required careful placement of printf statements and distinguishing between successful output vs error messages.
 2. **Detecting Command Not Found** — Since `execvp()` errors occur in a child process, we had to check if the captured output contained "Command not found" to print the appropriate [ERROR] message on the server side.
 3. **Consistent Error Handling** — Ensuring all error paths (pipe failure, fork failure, malloc failure, socket errors) were handled gracefully with proper [ERROR] messages and resource cleanup.
-

6 Division of Tasks

Task	Team Member
Socket implementation (server + client)	Yixuan Ma
Output capture via pipe + fork	Yixuan Ma
Length-prefix framing protocol	Yixuan Ma
Server output formatting ([INFO], [RECEIVED], etc.)	Uditi Sharma
Error handling and error messages	Uditi Sharma
Code comments	Uditi Sharma
Testing and verification	Both
Report writing	Both

7 References

- Beej's Guide to Network Programming: <https://beej.us/guide/bgnet/>
- Linux man pages: socket(2), bind(2), listen(2), accept(2), send(2), recv(2)
- Phase 1 shell implementation (our own code)